TCD3
Test Automation & Continuous Delivery

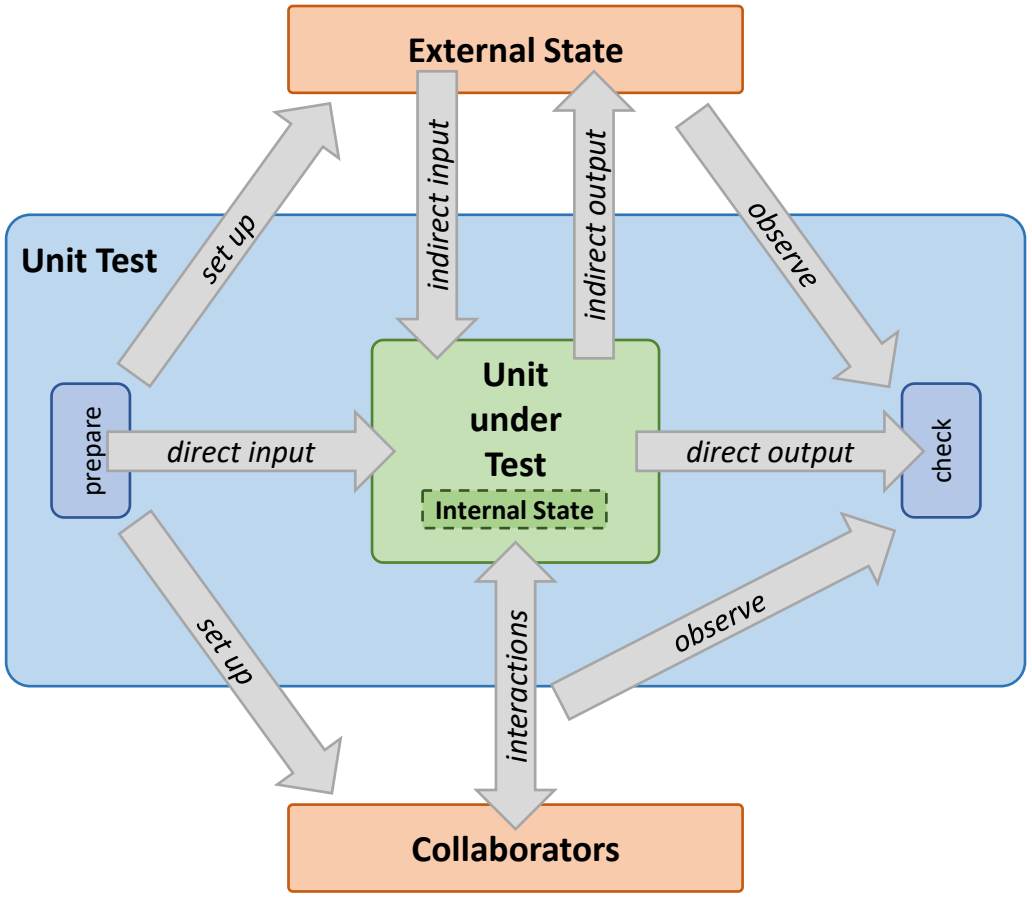
Test Doubles

S. Wagner

Anatomy of a Unit Test

- Direct input and direct output must be prepared and checked
- Internal state must be initialized and checked
- External state used by the unit under test must be set up and observed
- External collaborators called by the unit under test must be set up
- Interactions with external collaborators must be observed

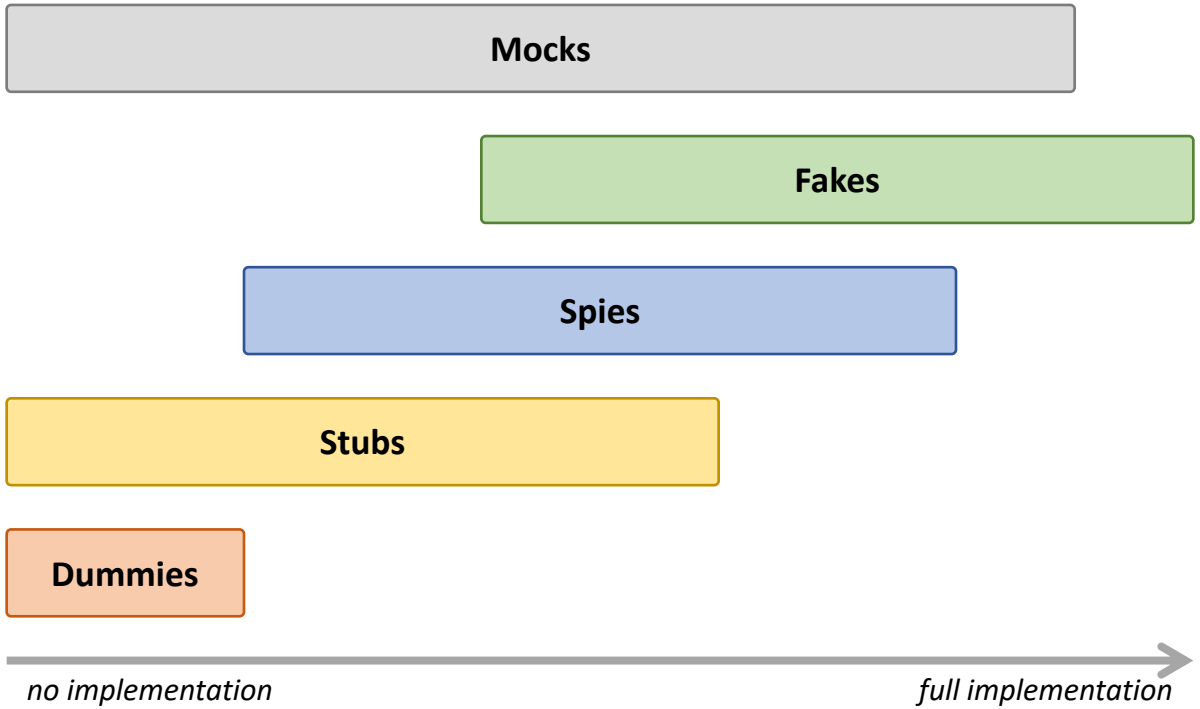
- AAA Pattern
  - Arrange
  - Act
  - Assert



Test Doubles

- Unit tests are executed in isolation
- Test doubles are needed to replace other units used by the unit under test
- Depending on their purpose, we differentiate between different kinds of test doubles:
  - **Dummy:** object which is required to fulfill an API, but which contains no implementation
  - **Stub:** object which returns hard-coded values
  - **Spy:** a stub which also tracks some interactions (partial mock)
  - **Fake:** object which replaces the original object with a simpler implementation (e.g., an in-memory database)
  - **Mock:** object which replaces the original object to track interactions
- Mocks are usually automatically generated using a mocking library
- Although you can create a mock that behaves like a dummy or a stub, be aware that mocks are intended to test behavior
- Extensive use of test doubles is an indicator for bad testability

Test Doubles



State Verification vs. Behavior Verification

- Unit test must check whether the test failed or not
- Verification can be done by checking states or by checking behavior

- State verification
  - expects a specific state after the unit under test was executed
  - uses assertions to compare current state and expected state
  - if they differ, the test fails
- Behavior verification
  - expects specific interactions between the unit under test and its collaborator
  - uses mocks or spies to track the interactions between the unit under test and its collaborator
  - if the expected interactions did not take place, the test fails

- State verification only checks the result, but not how this result has been achieved
- Behavior verification checks specific actions that lead to the desired result
- Behavior verification couples test code and production code more strongly (checks "how" and not "what")
- Behavior verification usually makes tests more brittle

Moq

https://github.com/moq/moq4



- Moq is an open-source mocking framework for .NET
  - development started in 2007 by Daniel Cazzulino
  - promoted by companies Clarius, Manas, and InSTEDD
  - in recent years main contributor is Dominique Schuppli
  - Moq supports Linq expression trees and lambda expressions

- Moq does not enforce the record/replay/verify pattern
  - expectations do not have to be defined upfront
  - stubbing and verification are clearly separated
  - Moq can verify anything after the mock object has been used

Moq vs. Rhino Mocks

Moq

```
public class MoqDemo {
  [Fact]
  public void TestMoq() {
    var mock = new Mock<IList>();
    mock.Setup(x => x[0]).Returns("one");  // setup
    mock.Setup(x => x[1]).Returns("two");
    IList list = mock.Object;

    Assert.Equal("one", list[0]);
    Assert.Equal("two", list[1]);
    list.Clear();

    mock.Verify(x => x.Clear());           // verify
  }
}
```

Rhino Mocks

```
public class RhinoMocksDemo {
  [Fact]
  public void TestRhinoMocks() {
    MockRepository mocks = new MockRepository();
    IList list = mocks.StrictMock<IList>();

    Expect.Call(list[0]).Return("one");  // record
    Expect.Call(list[1]).Return("two");
    Expect.Call(list.Clear);

    mocks.ReplayAll();                   // replay

    Assert.Equal("one", list[0]);
    Assert.Equal("two", list[1]);
    list.Clear();

    mocks.VerifyAll();                   // verify
  }
}
```

A First Mock Object

- Working with mock objects in Moq:
  1. create a mock object
  2. stub methods (if necessary)
  3. verify interactions (if necessary)
- remember the AAA-Pattern

```
public class PersonService {
  private readonly IPersonRepository repo;

  public PersonService(IPersonRepository repo) {
    this.repo = repo;
  }

  public void Register(string name, int age) {
    if (name == null) throw new ArgNullEx(nameof(name));
    if (string.IsNullOrEmpty(name)) throw new ArgEx("Inv. name");
    if (age < 16) throw new ArgEx("Person is too young");

    repo.CreatePerson(name, age);
  }

  public double GetAverageAge() => repo.ReadAllPersons()
                                       .Select(x => x.Age)
                                       .Average();
}
```

```
[Fact]
public void Register_WithValidNameAndAge_Succeeds() {
  var mock = new Mock<IPersonRepository>();
  PersonService sut = new(mock.Object);

  sut.Register("Alice", 25);

  mock.Verify(x => x.CreatePerson("Alice", 25));
}

[Fact]
public void Register_WithAgeTooYoung_ThrowsArgumentException() {
  var mock = new Mock<IPersonRepository>();
  PersonService sut = new(mock.Object);

  Assert.Throws<ArgumentException>(() => {
    sut.Register("Bob", 12);
  });

  mock.VerifyNoOtherCalls();
}

[Fact]
public void GetAverageAge_WhenPersonsRegistered_ReturnsAverageAge() {
  var mock = new Mock<IPersonRepository>();
  mock.Setup(x => x.ReadAllPersons()).Returns(new[] { new Person("a", 35),
                                                      new Person("b", 28) });
  double expected = (35 + 28) / 2f;
  PersonService sut = new(mock.Object);

  var result = sut.GetAverageAge();

  Assert.Equal(expected, result);
}
```

Stubs with Fixed Parameters

- Call Setup and Return to stub methods:
  - mock.Setup(…).Return(…)
- Parameters can be defined explicitly
- Any method which has not been stubbed returns a default value (null, 0, empty array, etc.)
- Only virtual methods can be stubbed

```
[Fact]
public void StubWithFixedParameter() {
  var mock = new Mock<IPersonRepository>();
  mock.Setup(x => x.ReadPersonById(1)).Returns(new Person("a"));
  mock.Setup(x => x.ReadPersonById(2)).Returns(new Person("b"));
  var rep = mock.Object;

  var personA = rep.ReadPersonById(1);
  var personB = rep.ReadPersonById(2);
  var personC = rep.ReadPersonById(3);  // not stubbed -> returns null

  Assert.Equal("a", personA.Name);
  Assert.Equal("b", personB.Name);
  Assert.Null(personC);
}
```

Stubs with any Parameters

- Argument matchers can be used for any parameters
  - It.IsAny, It.Is, It.IsIn, It.IsNotIn, It.IsInRange, It.IsNotNull, It.IsRegex, …

```
[Fact]
public void StubWithAnyParameter() {
  var mock = new Mock<IPersonRepository>();
  mock.Setup(x => x.ReadPersonById(It.IsAny<int>()))
      .Returns(new Person("a"));
  var rep = mock.Object;

  var personA = rep.ReadPersonById(1);
  var personB = rep.ReadPersonById(2);
  var personC = rep.ReadPersonById(3);

  Assert.Equal("a", personA.Name);
  Assert.Equal("a", personB.Name);
  Assert.Equal("a", personC.Name);
}
```

- It.Is<…>(Func<…> match) allows to implement complex custom matchers

```
It.Is<int>(i => i % 2 == 0)
```

Stubs with Multiple Returns

- Use SetupSequence to provide multiple return values
- If no more return values are available, default values (null, 0, etc.) are returned

```
[Fact]
public void StubWithReturnSequence() {
  var mock = new Mock<IPersonRepository>();
  mock.SetupSequence(x => x.ReadPersonById(It.IsAny<int>()))
      .Returns(new Person("a"))
      .Returns(new Person("b"));
  var rep = mock.Object;

  var personA = rep.ReadPersonById(1);
  var personB = rep.ReadPersonById(2);
  var personC = rep.ReadPersonById(3);  // returns null

  Assert.Equal("a", personA.Name);
  Assert.Equal("b", personB.Name);
  Assert.Null(personC);
}
```

Multiple Matching Stubs

- If multiple stubs match, the latest stub is used
  - therefore, stubs defined later in the code override previous stubs
  - if a default stub is defined for each test case (e.g., in the constructor), it can be redefined in the test method

```
[Fact]
public void MultipleMatchingStubs() {
  var mock = new Mock<IPersonRepository>();
  mock.Setup(x => x.ReadPersonById(It.IsAny<int>()))
      .Returns(new Person("x"));
  mock.SetupSequence(x => x.ReadPersonById(It.IsAny<int>()))
      .Returns(new Person("a"))
      .Returns(new Person("b"));
  mock.Setup(x => x.ReadPersonById(3))
      .Returns(new Person("c"));
  var rep = mock.Object;

  var personA = rep.ReadPersonById(1);
  var personB = rep.ReadPersonById(2);
  var personC = rep.ReadPersonById(3);

  Assert.Equal("a", personA.Name);
  Assert.Equal("b", personB.Name);
  Assert.Equal("c", personC.Name);
}
```

Stubs Using Arguments and Throwing Exceptions

- Invocation argument can be accessed to build specific return values

```
[Fact]
public void StubWithUseOfArgument() {
  var mock = new Mock<IPersonRepository>();
  mock.Setup(x => x.ReadPersonById(It.IsAny<int>()))
      .Returns((int i) => { return new Person(i, "a", 25); });
  var rep = mock.Object;

  var personA = rep.ReadPersonById(1);
  var personB = rep.ReadPersonById(2);
  var personC = rep.ReadPersonById(3);

  Assert.Equal(1, personA.Id);
  Assert.Equal(2, personB.Id);
  Assert.Equal(3, personC.Id);
}
```

- Stubbed methods can also throw exceptions

```
[Fact]
public void StubThrowsException() {
  var mock = new Mock<IPersonRepository>();
  mock.Setup(x => x.ReadPersonById(-1)).Throws<ArgumentException>();
  var rep = mock.Object;

  Assert.Throws<ArgumentException>(() => {
    rep.ReadPersonById(-1);
  });
}
```

Verify for Fixed or any Parameters

- Fixed parameter values or argument matchers can also be used for verification

```
[Fact]
public void VerifyForFixedParameter() {
  var mock = new Mock<IPersonRepository>();
  mock.Setup(x => x.ReadPersonById(It.IsAny<int>()))
      .Returns(new Person("a"));
  var rep = mock.Object;

  var personA = rep.ReadPersonById(0);

  mock.Verify(x => x.ReadPersonById(0));
}
```

```
[Fact]
public void VerifyForAnyParameter() {
  var mock = new Mock<IPersonRepository>();
  mock.Setup(x => x.ReadPersonById(It.IsAny<int>()))
      .Returns(new Person("a"));
  var rep = mock.Object;

  var personA = rep.ReadPersonById(0);

  mock.Verify(x => x.ReadPersonById(It.IsAny<int>()));
}
```

Verify by Number of Calls

- Verify how often an interaction has occurred
- VerifyNoOtherCalls checks if any additional interactions took place
- Be careful, do not oververify your tests!

```
[Fact]
public void VerifyByNumberOfCalls() {
  var mock = new Mock<IPersonRepository>();
  mock.Setup(x => x.ReadPersonById(It.IsAny<int>()))
      .Returns(new Person("a"));
  var rep = mock.Object;

  var personA = rep.ReadPersonById(0);
  var personB = rep.ReadPersonById(1);

  mock.Verify(x => x.ReadPersonById(It.IsAny<int>()), Times.Exactly(2));
  mock.Verify(x => x.ReadPersonById(It.IsAny<int>()), Times.AtLeastOnce);
  mock.Verify(x => x.ReadPersonById(It.IsAny<int>()), Times.AtLeast(2));
  mock.Verify(x => x.ReadPersonById(It.IsAny<int>()), Times.AtMost(2));
  mock.Verify(x => x.ReadPersonById(2),               Times.Never);
}
```

```
[Fact]
public void VerifyNoMoreInteractionsOccurred() {
  var mock = new Mock<IPersonRepository>();
  mock.Setup(x => x.ReadPersonById(It.IsAny<int>()))
      .Returns(new Person("a"));
  var rep = mock.Object;

  var personA = rep.ReadPersonById(0);

  mock.Verify(x => x.ReadPersonById(0));
  mock.VerifyNoOtherCalls();
}
```

Strict Mocks vs. Loose Mocks

- Per default mocks return default values for all methods that have not been stubbed (i.e., MockBehavior.Loose)
- If MockBehavior.Strict is set, calling methods which have not been stubbed throws an exception

```
[Fact]
public void LooseMockDemo() {
  var mock = new Mock<IList>(MockBehavior.Loose);
  mock.Setup(x => x[0]).Returns(1);
  IList list = mock.Object;

  var value1 = list[0];
  var value2 = list[1];  // not stubbed -> returns null (default value)

  Assert.Equal(1, value1);
  Assert.Null(value2);
}
```

```
[Fact]
public void StrictMockDemo() {
  var mock = new Mock<IList>(MockBehavior.Strict);
  mock.Setup(x => x[0]).Returns(1);
  IList list = mock.Object;

  var value1 = list[0];
  var value2 = list[1];  // not stubbed -> throws exception

  Assert.Equal(1, value1);
  Assert.Equal(0, value2);
}
```

Mocks vs. Spies

- Mocks
  - new objects which do not contain any implementation
  - stubs can be added for virtual methods
  - can also be created for interfaces

- Spies
  - objects of existing classes which are extended with behavior tracking
  - objects contain default implementation
  - virtual methods can be replaced by stubs
  - can only be created for instantiable types
  - CallBase defines if the base implementation of a method is called, if no stub is defined
    - defaults to false

```
[Fact]
public void MockDemo() {
  var mock = new Mock<IList>();
  IList list = mock.Object;

  list.Add(1);
  var result = list[0];

  Assert.Null(result);
}
```

```
[Fact]
public void SpyDemo() {
  var mock = new Mock<ArrayList> { CallBase = true };
  IList list = mock.Object;

  list.Add(1);
  var result = list[0];

  Assert.Equal(1, result);
}
```

Dangers and Pitfalls

- Oververification
  - each verification of a mock object couples the test to the internal implementation of the SUT
  - tests become brittle and sensitive to change
  - refactoring might break tests which should still be green
  - avoid too many and too precise verifications

- Mocks everywhere
  - creating mocks is easy, maybe too easy
  - if a real class exists that is sufficient for the current test, use it
  - do not mock types that you do not own

- Transitive mocking
  - be alarmed if a mock returns a mock
  - transitive verification of interactions usually is not reasonable

Test Driven Development

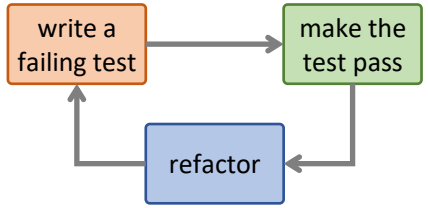
- 3 laws of TDD

> ***You may not write production code until you have written a failing unit test.***

> ***You may not write more of a unit test than is sufficient to fail, and not compiling is failing.***

> ***You may not write more production code than is sufficient to pass the currently failing test.***

- Tests and production code are developed together in a very tight loop
- Thinking about and writing tests first has some benefits
  - you must express first, which functionality and API you want to offer
  - your production code will be testable and therefore better designed
  - your production code will be fully covered by tests



- However, pure TDD is hard and a bit unrealistic, but the principle is valuable

Better Bad Tests than No Tests?

- Test suite has to evolve together with the system
- If tests are bad …
  - bad tests make it harder to keep production code and test code in sync
  - tests consume more and more time
  - keeping up the test suite becomes unaffordable
  - test are silenced
  - finally test suite is abandoned
  - without tests, more defects arise
  - more defects lead to a fear of change
  - without change, code starts to rot
- Bad tests can be worse than no tests at all
- Test code deserves (and requires) the same level of care as production code
- Test code is not second-class code

Test Smells

Code Smells

- Obscure tests
  - test cannot be understood at a glance
- Conditional test logic
  - tests contain conditional code that may or may not be executed
- Hard-to-Test Code
  - testability of the unit under test is bad
- Test Code Duplication
  - same test code is repeated multiple times
- Test Logic in Production
  - production code contains routines which are only for testing

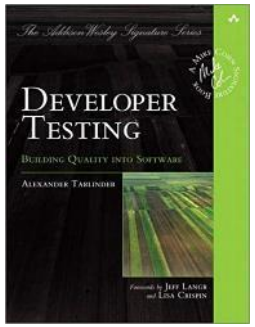
Behavior Smells

- Assertion roulette
  - it is hard to tell which assertion caused a test to fail
- Erratic tests
  - tests seem to pass/fail non-deterministically
- Fragile tests
  - tests fail although they should not
- Frequent Debugging
  - manual debugging is required to find out why a test failed
- Manual intervention
  - tests require manual action when they are run
- Slow tests
  - tests take to long to run

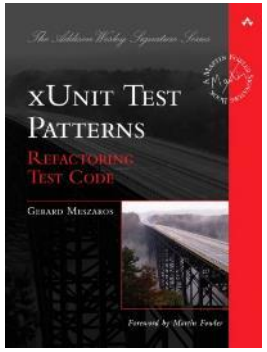
Literature



B. Garcia:
Mastering Software Testing with JUnit 5
Packt Publishing



A. Tarlinder:
Developer Testing: Building Quality into Software
Addison Wesley



G. Meszaros:
xUnit Test Patterns: Refactoring Test Code
Addison Wesley